Java Collections Framework Cheat Sheet (Part 1)

Core Hierarchy, List Implementations, and Set Collections

Senior Technical Consultant
Contact: java.kumar.arun@gmail.com

FRAMEWORK ARCHITECTURE

The **Java Collections Framework (JCF)** provides a unified architecture to store, manipulate, and manage groups of objects efficiently using built-in interfaces.

Root Interface Hierarchy:

```
Iterable<E> (Root)
   └── Collection<E>
        ├── List<E> (Ordered, Duplicates OK)
        ├── Set<E> (Unordered, Unique elements)
        └── Queue<E> (FIFO / Processing order)
```

Note: The `Map<K,V>` interface is independent but fully integrated into the framework architecture.

GENERICS TYPE SAFETY

Always specify the wrapper class type inside angle brackets `<T>` to catch illegal data additions at compile-time instead of hitting a runtime error.

```
// Enforces only Integers inside the collection
List<Integer> scores = new ArrayList<>();
scores.add(95);
// scores.add("Top"); // Compile-time Error
```

LIST INTERFACE IMPLEMENTATIONS

An ordered collection where indices map positions. Duplicate items are completely allowed.

ArrayList (Dynamic Resizing Array):

Best for rapid random access lookups via $O(1)$ loops. Expensive shifts on middle insertions: $O(n)$.

```
import java.util.ArrayList;
List<String> list = new ArrayList<>();
list.add("Flutter");
list.add("Python");
System.out.println(list.get(0)); // Flutter
```

LinkedList (Doubly Linked Structure):

Fast sequentially chained node insertions or deletions: $O(1)$. Linear search lookups: $O(n)$.

```
import java.util.LinkedList;
List<Integer> link = new LinkedList<>();
link.add(10);
```

SET INTERFACE IMPLEMENTATIONS

A collection that contains absolutely no duplicate elements. Models the mathematical set abstraction wrapper.

HashSet (Hash Table Storage):

Offers superb execution speed performance for insertions, deletions, and searches: $O(1)$. Makes no performance or order preservation guarantees over time.

```
import java.util.HashSet;
Set<String> set = new HashSet<>();
set.add("Delhi");
set.add("Delhi"); // Duplicate addition rejected
safely
System.out.println(set.size()); // Output: 1
```

TreeSet (Sorted Balanced Tree):

Maintains element order using natural sorting values or a custom comparator function: $O(\log n)$ operations.

```
import java.util.TreeSet;
Set<Integer> sortedSet = new TreeSet<>();
sortedSet.addAll(java.util.List.of(30, 10, 20));
System.out.println(sortedSet); // [10, 20, 30]
```

Java Collections Framework Cheat Sheet (Part 2)

Map Mappings, Queue Pipelines, Iteration Mechanics, and Utility Classes

Senior Technical Consultant
Contact: java.kumar.arun@gmail.com

MAP INTERFACE (KEY-VALUE PAIRS)

Maps relate unique identifiers directly to specific value references. They do not implement `Collection`.

HashMap (Hash Table backed):

Unordered access pathing: $O(1)$ tracking speed. Allows a single null key alongside multiple null values.

```
import java.util.HashMap;
Map<String, Integer> map = new HashMap<>();
map.put("Salary", 1200000);
map.put("Bonus", 50000);

System.out.println(map.get("Salary")); // 1200000
// Loop over keys and values
for (Map.Entry<String, Integer> entry :
map.entrySet()) {
    System.out.println(entry.getKey() + ": " +
entry.getValue());
}
```

TreeMap (Red-Black Tree backend):

Keys are sorted explicitly in ascending alphabetical or numeric order: $O(\log n)$ structure.

QUEUE & DEQUE INTERFACES

Designed explicitly to hold element items in sequence priority order prior to localized thread consumption routines.

PriorityQueue (Binary Heap Implement):

Items are systematically extracted depending on sorting priority parameters, not simple arrival order.

```
import java.util.PriorityQueue;
Queue<Integer> pq = new PriorityQueue<>();
pq.add(40); pq.add(10);
System.out.println(pq.peek()); // 10 (Lowest item
out first)
```

ArrayDeque (Double-Ended Queue):

Resizable pointer ring providing high performance head and tail operations. Ideal to implement custom Stacks or FIFO lines.

```
import java.util.ArrayDeque;
Deque<String> stack = new ArrayDeque<>();
stack.push("Frame1"); // Stack Push
String top = stack.pop(); // Stack Pop
```

TRAVERSING COLLECTIONS

The Enhanced For-Each Loop:

```
List<String> items = List.of("A", "B", "C");
for (String item : items) {
    System.out.println(item);
}
```

The Safe Iterator Interface:

The only safe way to modify or remove list elements cleanly mid-flight without throwing a `ConcurrentModificationException`.

```
import java.util.Iterator;
Iterator<String> it = mutableList.iterator();
while (it.hasNext()) {
    if (it.next().equals("B")) {
        it.remove(); // Structural Safe Deletion
    }
}
```

THE COLLECTIONS UTILITY CLASS

Provides robust static algorithms to optimize collection manipulation handling:

```
import java.util.Collections;
Collections.sort(myList);     // Sort List
Collections.reverse(myList);  // Flip elements
order
Collections.shuffle(myList);  // Randomize layouts
```